

COMP6237 Data Mining

Lecture 4: Embedding Data (Dimensionality Reduction II)

Zhiwu Huang

Zhiwu.Huang@soton.ac.uk

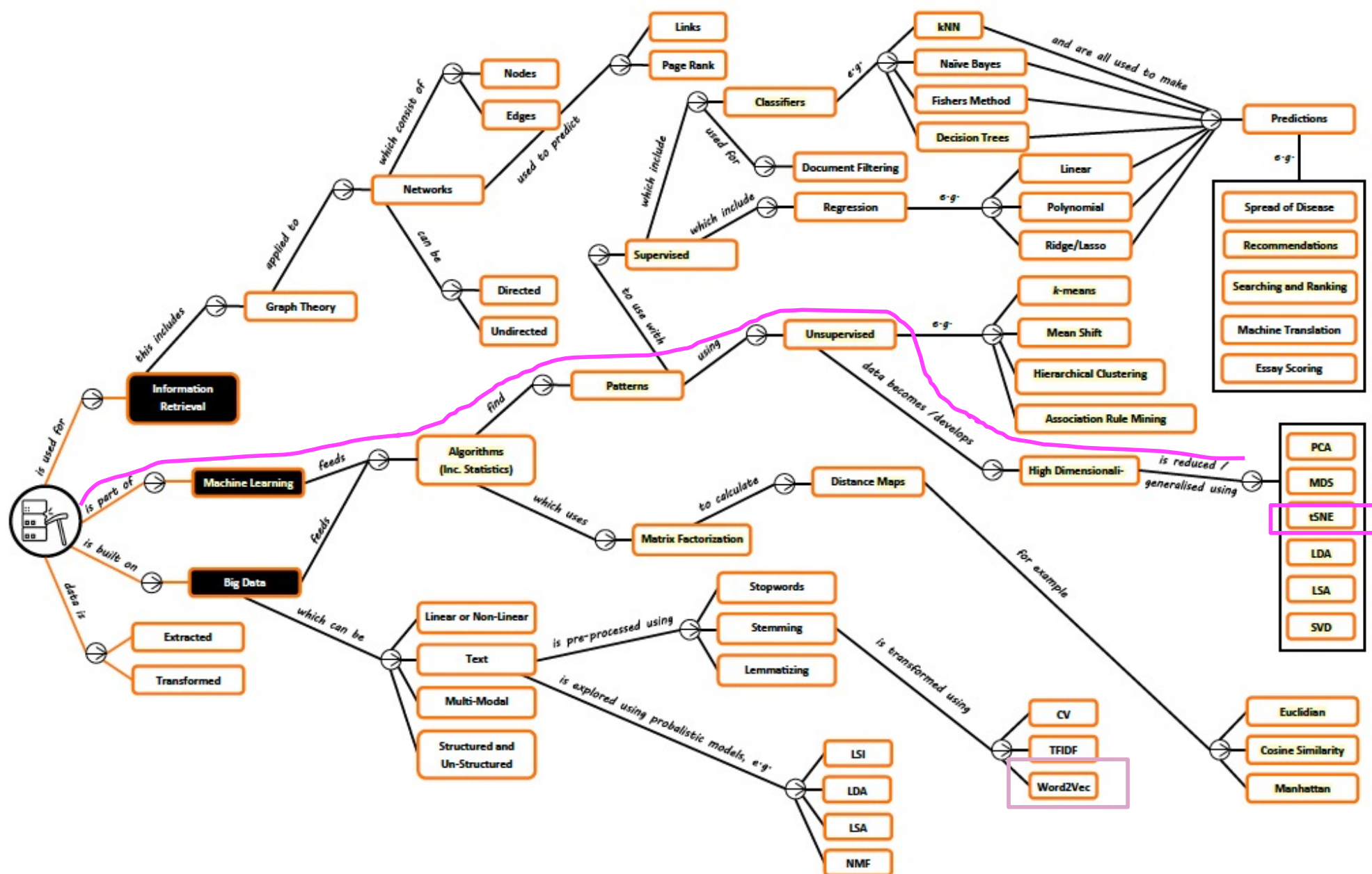
Lecturer (Assistant Professor) @ VLC of ECS
University of Southampton

Lecture slides available here:

<http://comp6237.ecs.soton.ac.uk/zh.html>

(Thanks to Prof. Jonathon Hare and Dr. Jo Grundy for providing the lecture materials used to develop the slides.)

Dimensionality Reduction II – Roadmap



Dimensionality Reduction I

- when data structure is simple and linear

Each giraffe has height and weight, so each one is a point in 2D.

We saw that height and weight are correlated.

PCA found a direction that captures most of that variation.

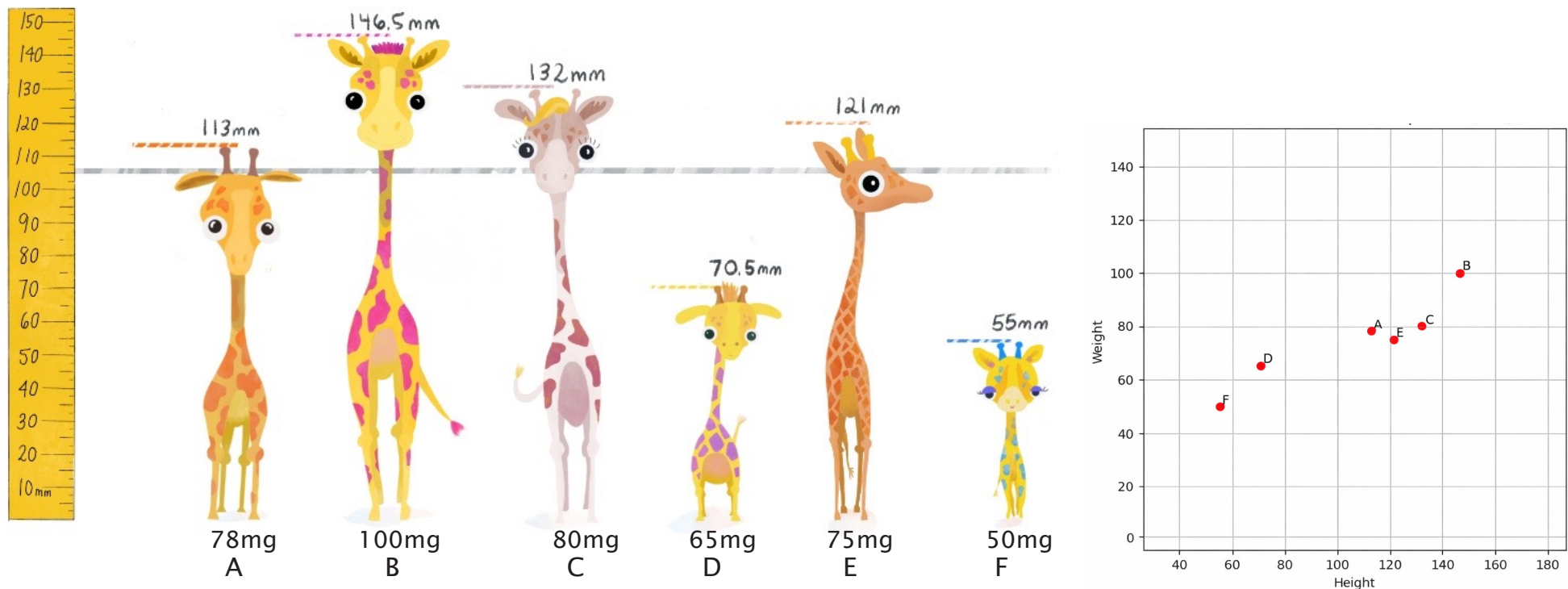


Image source: https://tinystats.github.io/teacups-giraffes-and-statistics/04_variance.html

Dimensionality Reduction I

- when data structure is simple and linear

PCA works well here because the structure is simple and linear.

One direction explains most of the variation.

Projecting onto that line keeps most of the information.

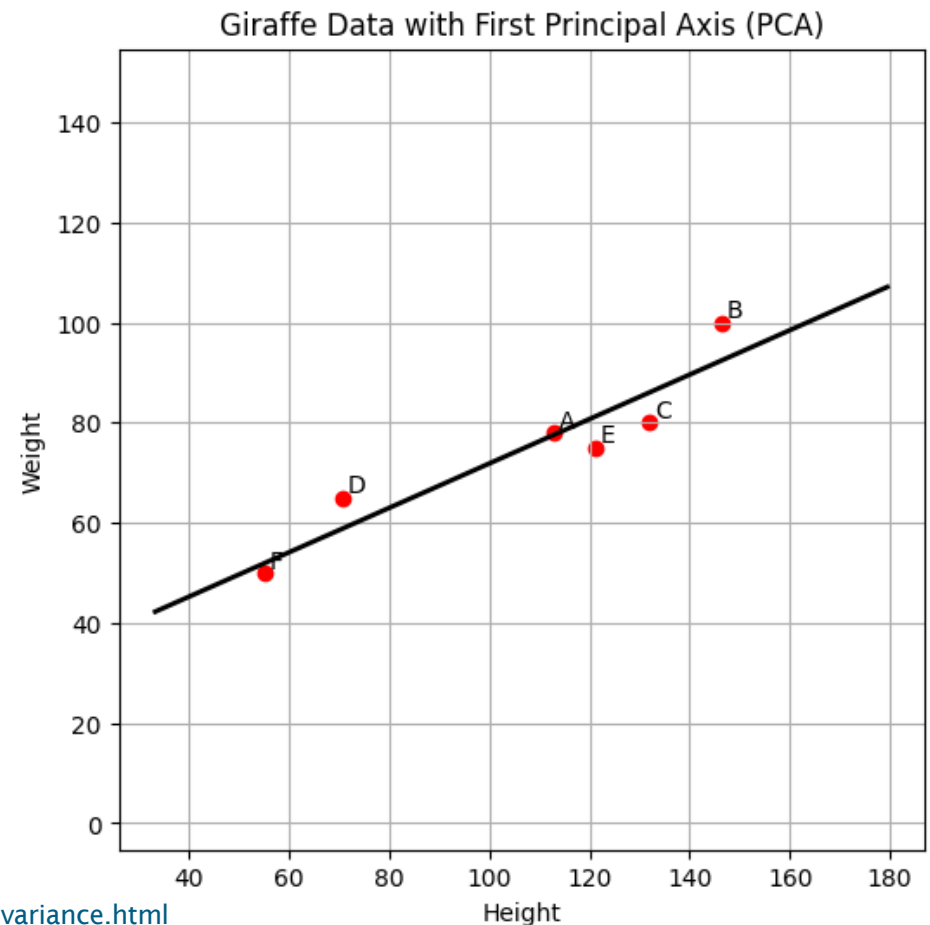
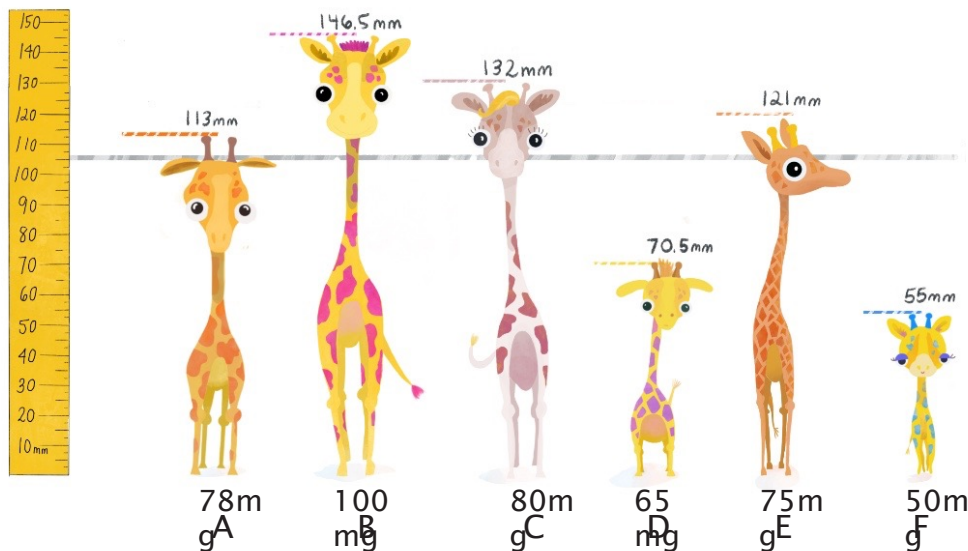


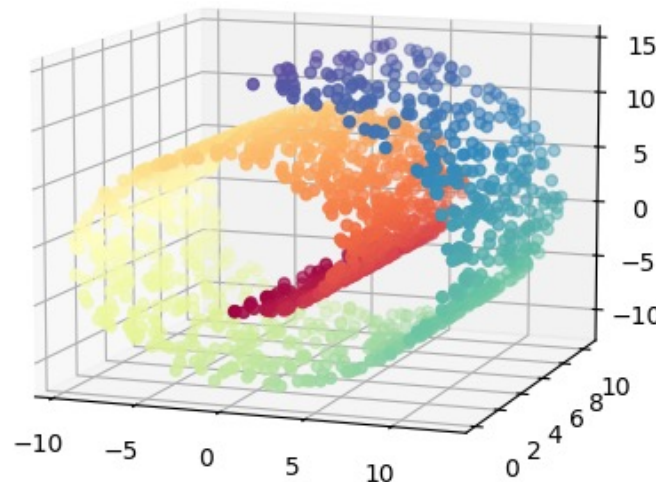
Image source: https://tinystats.github.io/teacups-giraffes-and-statistics/04_variance.html

Dimensionality Reduction II

- what if data structure gets more complex?

But what if giraffes from groups not just by size?

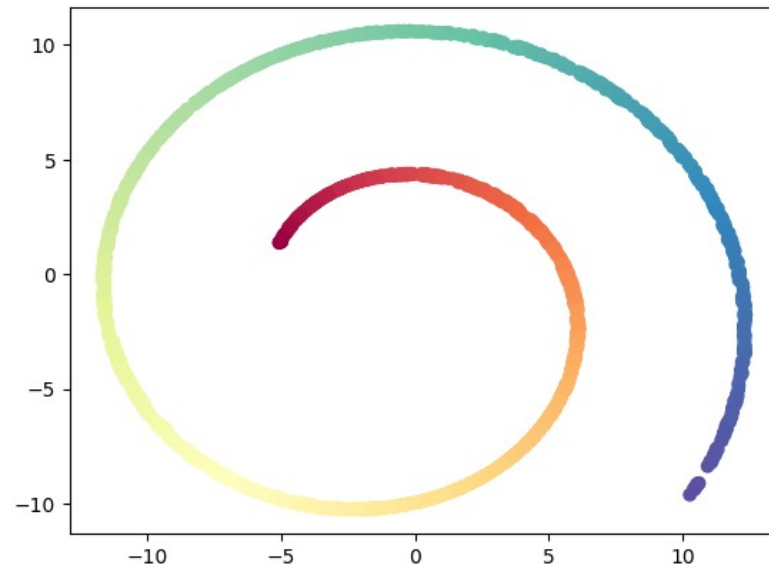
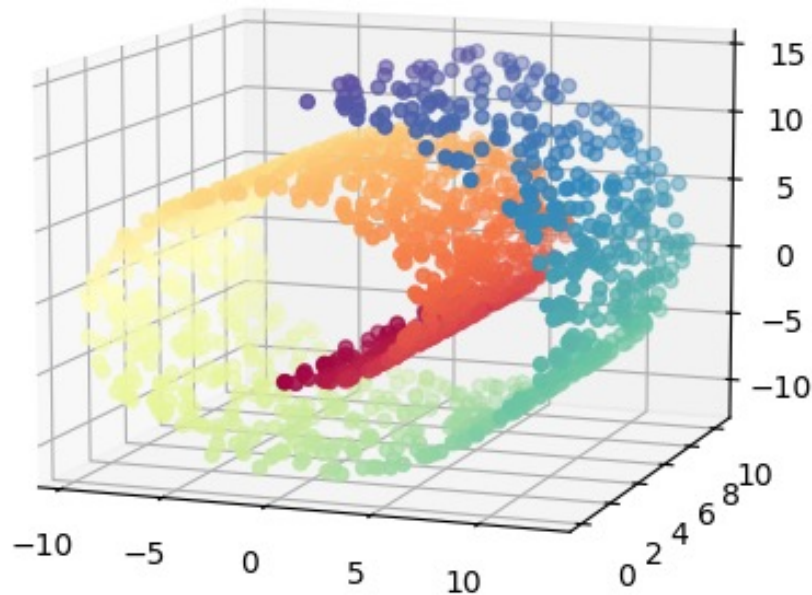
- Maybe by habit, age, or health
- Some giraffes might be similar locally, even if they differ globally



Swiss-roll data

Dimensionality Reduction II

- what if data structure gets more complex?

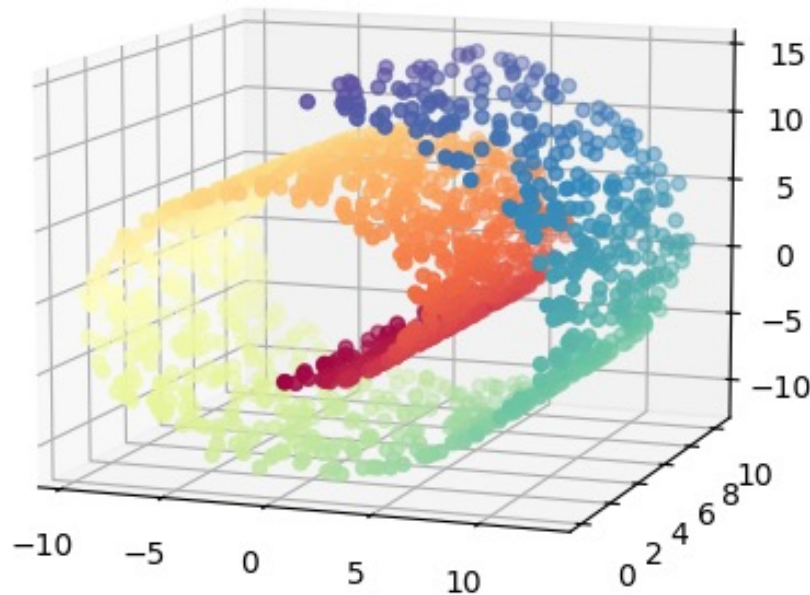


The Swiss roll is a **non-linear manifold**.
Locally, points that are next to each other really are similar,
but **globally the structure is curved and twisted**.

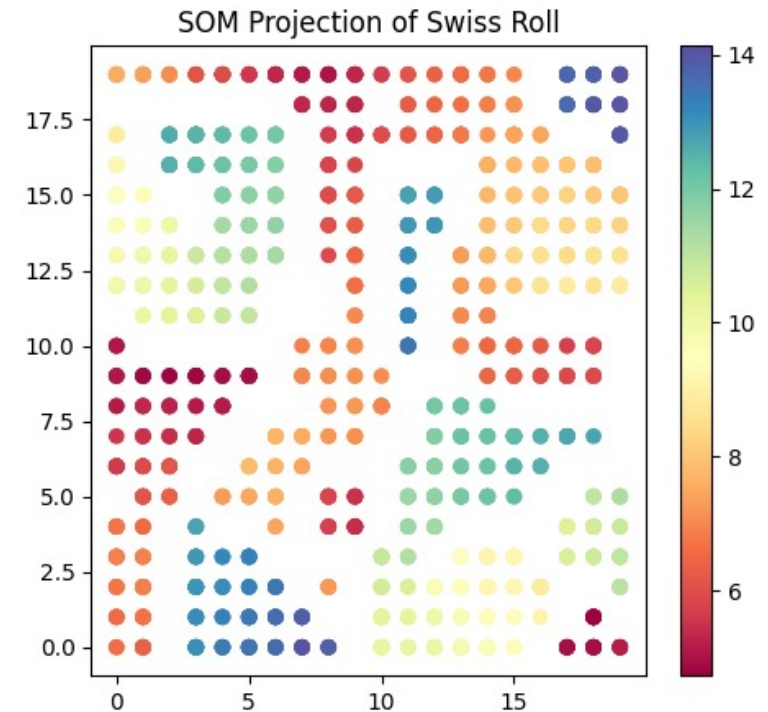
PCA only looks for **one global linear direction** that explains the most variance.
it does not preserve neighbourhoods or topology.

Dimensionality Reduction II

- what if data structure gets more complex?



The Swiss roll is a **non-linear manifold**.
Locally, points that are next to each other really are similar,
but **globally the structure is curved and twisted**.

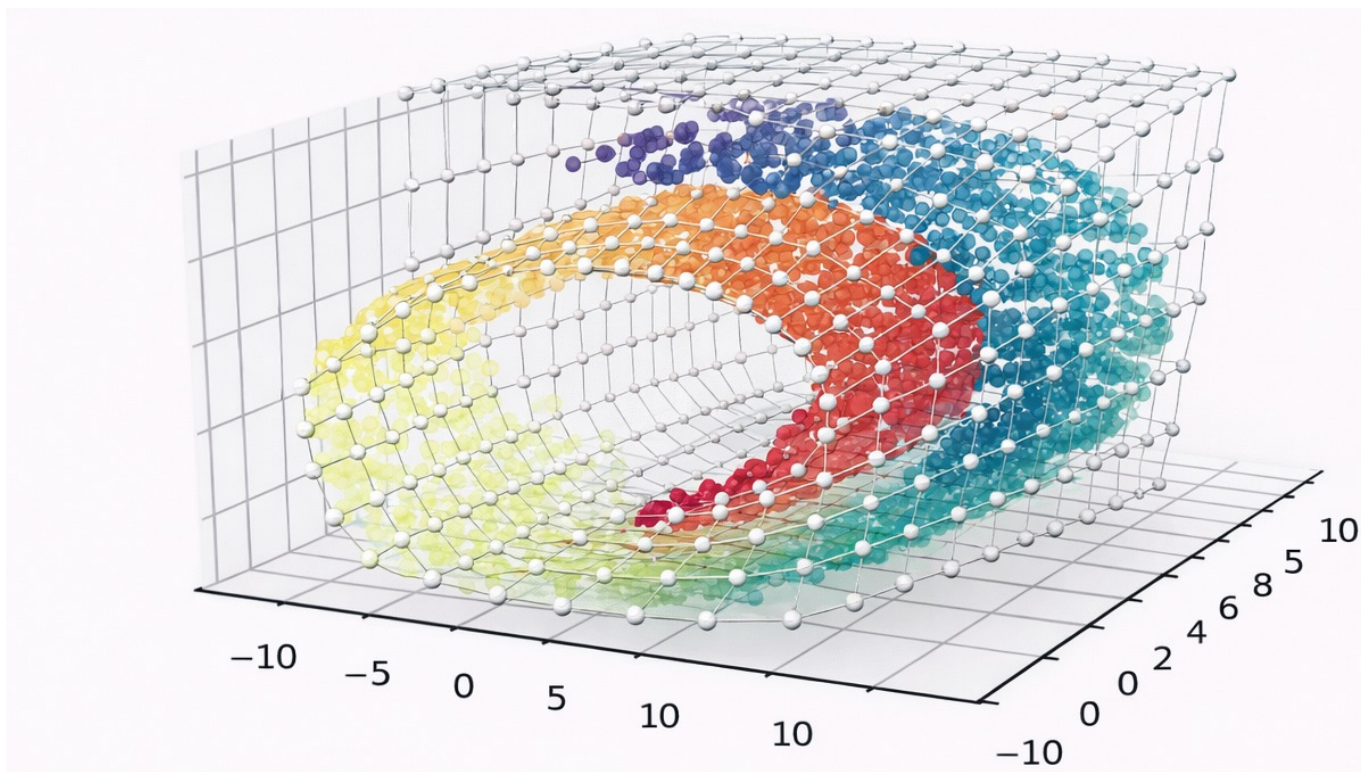


Better?

- **Topological structure is preserved**
- Local relationships are kept
- Neighbourhoods are very stable

From a *topology* perspective, this is **very good**.

Embedding Data – Self-Organising Maps



Now imagine we drop a flexible 2D grid into this 3D space

Each grid point represents a prototype data point

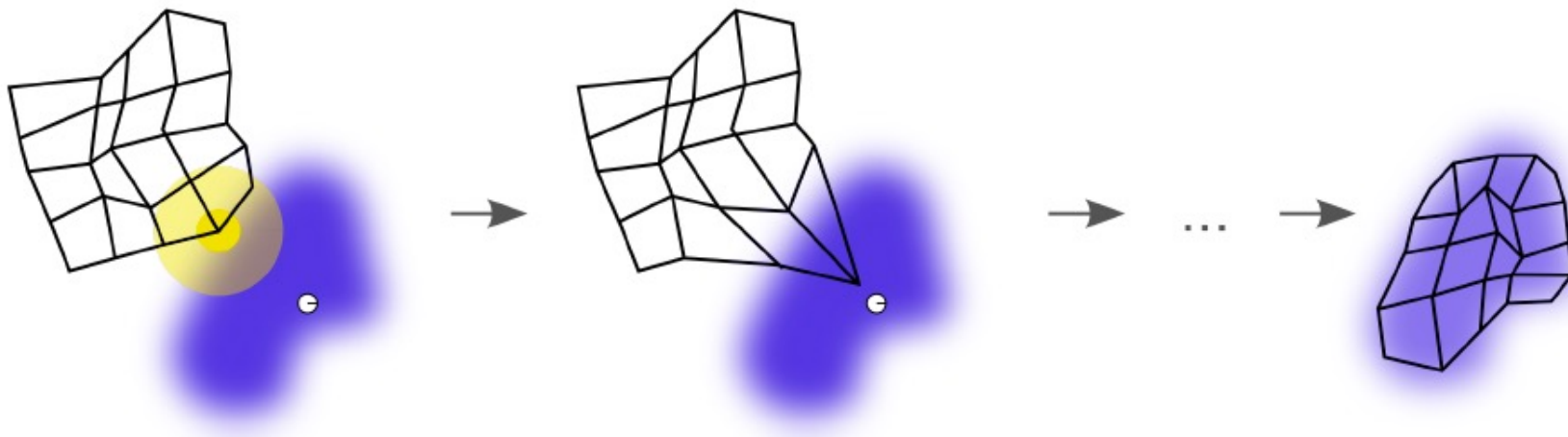
Embedding Data – Self-Organising Maps

SOMs: two phases

- ▶ training
- ▶ mapping

To start training, the set of nodes each has a random starting position defined in the feature space.

This is then updated by taking one feature vector, finding which unit is the *best matching unit* (BMU) then moving that unit and, to a lesser extent, its neighbours, closer to that data point



Embedding Data – Self-Organising Maps

Algorithm 1: Training Self-Organising Maps

Data: N data points with d dimensional feature vectors X_i

$i = 1 \dots N$, number of iterations λ

$\mathbf{w}$ = randomly initialise $n \times m$ units with weight vector ;

$t = 0$;

while $t < \lambda$ **do**

for each x_i **do**

$BMU = w_{nm}$ with min distance;

 Update BMU and its neighbours by moving closer to x_i ;

end

$t = t + 1$

end

Embedding Data – Self-Organising Maps

To update the weight vector $\mathbf{w}$

$$\mathbf{w}(t + 1) = \mathbf{w}(t) + \theta(u, v, t)\alpha(t)(x_i - \mathbf{w}(t))$$

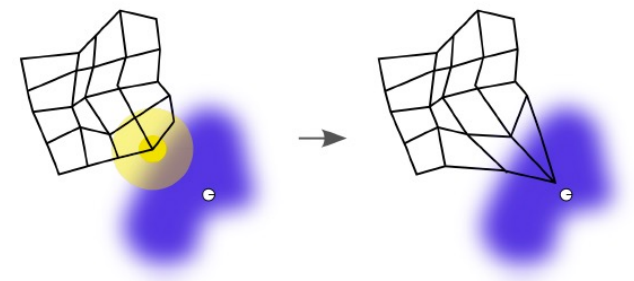
where θ is the neighbourhood weighting function (usually Gaussian)

e.g., $\theta = \begin{cases} 1 & \text{if the node is the winner} \\ 0.5 & \text{if the node is immediate neighbour of the winner} \\ 0 & \text{otherwise} \end{cases}$

α is the learning rate

x_i is the input vector

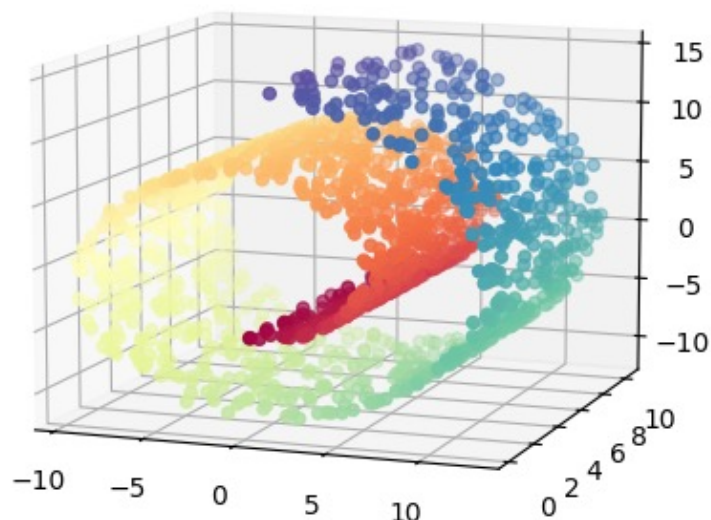
u is the winner node, and v is the node whose weight is $\mathbf{w}$



Both the learning rate and the neighbourhood weighting function get smaller over time

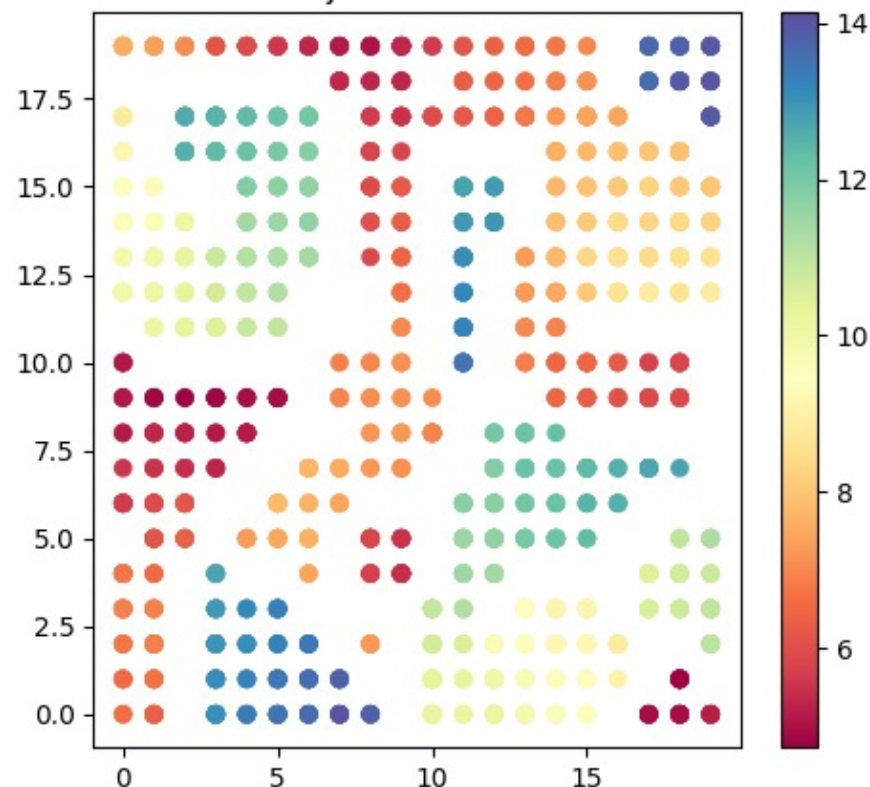
Embedding Data – Self-Organising Maps

Original Swiss Roll



The data in 3D has a clear structure

SOM Projection of Swiss Roll



Using SOM, it can much better group the data

ipynb mean centering demo <https://github.com/zhiwu-huang/Data-Mining-Demo-Code-18-19>

Embedding Data – Stochastic Neighbour Embedding

Stochastic Neighbour Embedding (SNE)

SNE optimises the distribution of data

Aims to make the distribution of the projected data in low dimensional space close to the actual distribution in high dimensional space

Embedding Data – Stochastic Neighbour Embedding

To calculate the source distribution:

We define a conditional probability that high-dimensional x_i would pick x_j as a neighbour if the neighbours were picked in proportion to their probability density under a Gaussian centred at x_i

$$p_{j|i} = \frac{\exp^{-\|x_i - x_j\|^2 / 2\sigma_i^2}}{\sum_{k \neq i} \exp^{-\|x_i - x_k\|^2 / 2\sigma_i^2}}$$

The SNE algorithm chooses σ for each data point such that smaller σ is chosen for points in dense parts of the space, and larger σ is chosen for points in sparse parts

Embedding Data – Stochastic Neighbour Embedding

To calculate the target distribution:

Define a conditional probability that low-dimensional y_i would pick y_j as a neighbour if the neighbours were picked in proportion to their probability density under a Gaussian centred at y_i

$$q_{j|i} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq i} \exp(-\|y_i - y_k\|^2)}$$

In this space we assume the variance of all Gaussians is $1/\sqrt{2}$ in this space

Embedding Data – Stochastic Neighbour Embedding

To measure the difference between two probability distributions we use the *Kullback-Leibler* (KL) Divergence:

$$D_{KL}(P|Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

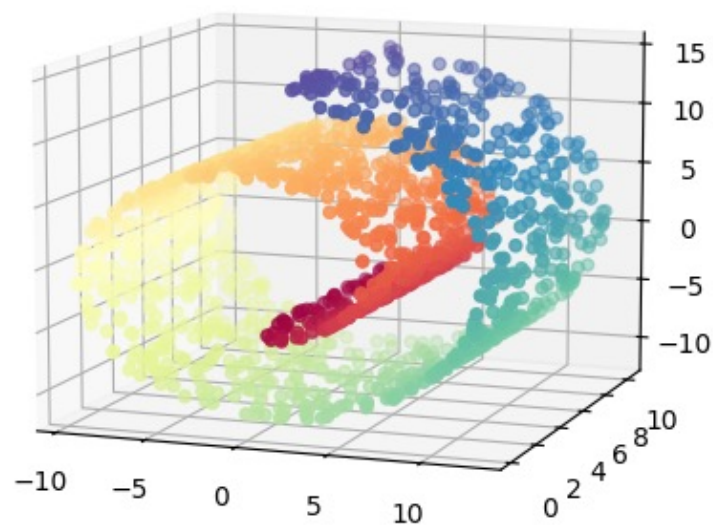
The cost function is the KL divergence summed over all data points

$$C = \sum_i \sum_j p_{i|j} \log \frac{p_{j|i}}{q_{j|i}}$$

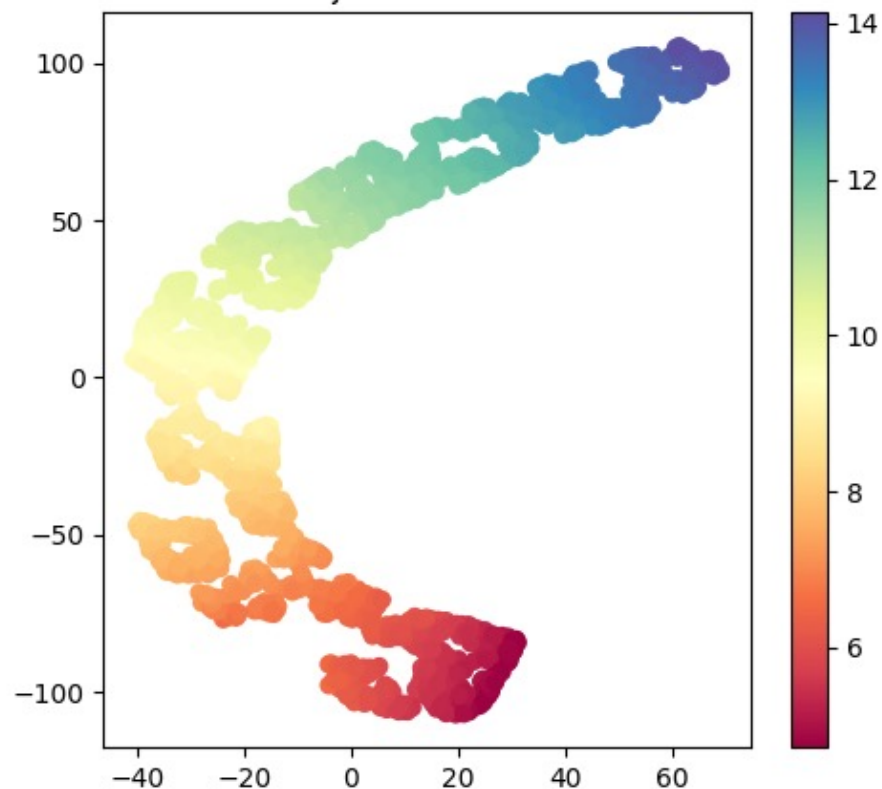
C can be minimised using *gradient descent* - but..
difficult to optimise, leads to crowded visualisations, big clumps of data together in the center

Embedding Data – Stochastic Neighbour Embedding

Original Swiss Roll

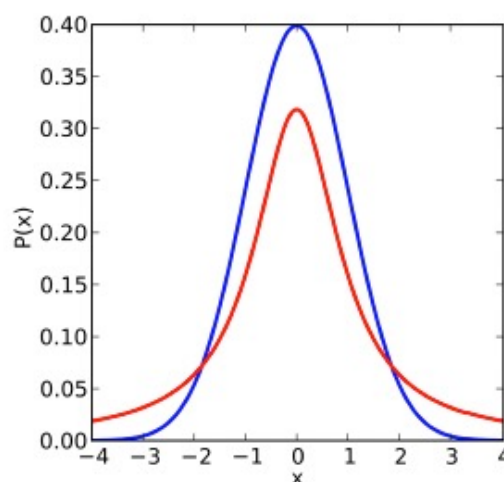


SNE Projection of Swiss Roll



Embedding Data – t- Stochastic Neighbour Embedding

In 2008 Maaten and Hinton came up with a way to improve SNE, by replacing the Gaussian distribution for the lower dimensional space with a Student's t distribution.



Student's t distribution in red, Gaussian distribution in blue

The Student's t distribution has a much longer tail, helps avoid clumping in the middle.

Embedding Data – t- Stochastic Neighbour Embedding

The cost function is also modified, making gradients simpler, so faster to compute

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2N}$$

To alleviate crowding using Student's t distribution in the lower dimensional space:

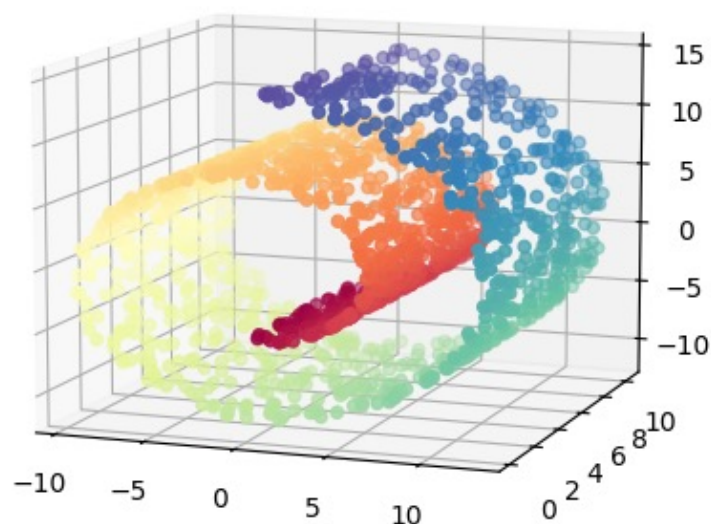
$$q_{ij} = \frac{(1 + ||x_i - x_j||)^{-1}}{\sum_{k \neq i} (1 + ||x_i - x_k||)^{-1}}$$

we use 1 degree of freedom with the Student's t distribution, equivalent to a Cauchy distribution

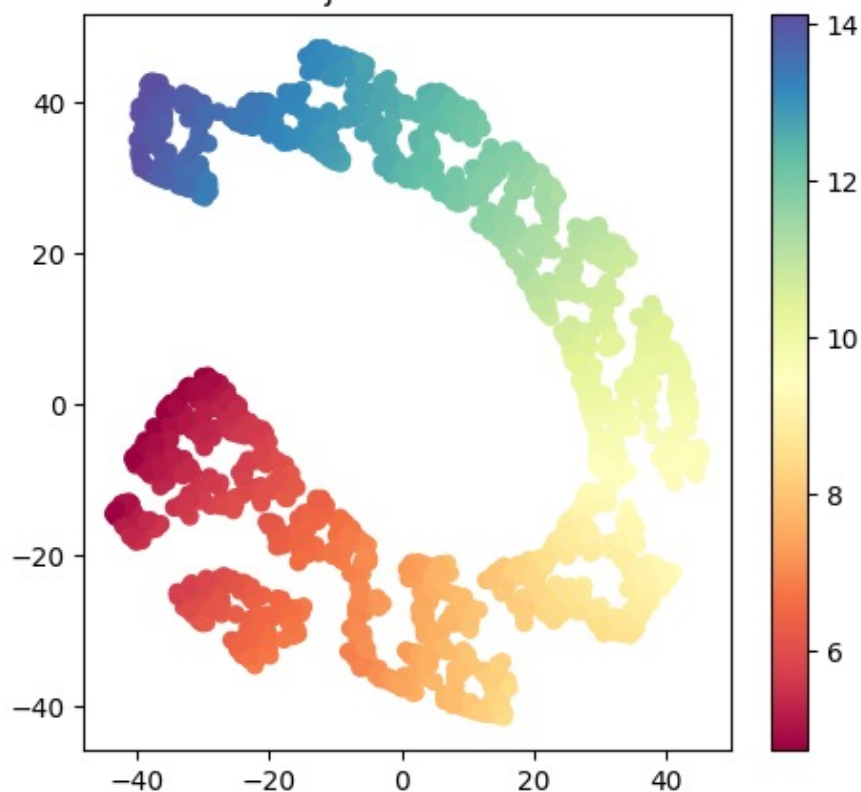
Embedding Data – t- Stochastic Neighbour Embedding

For the Swiss Roll data:

Original Swiss Roll



t-SNE Projection of Swiss Roll



ipy nb mean centering demo <https://github.com/zhiwu-huang/Data-Mining-Demo-Code-18-19>

Embedding Data – Summary

	Key Idea	Pros	Cons
SOM	Uses competitive learning to map high-dimensional data onto a grid of neurons (usually 2D) while preserving topology.	- Good for clustering and visualization - Preserves topological relationships - Works well with unsupervised learning	- Can be slow for large datasets - Grid size affects performance - Not optimal for non-topological structures
SNE	Converts high-dimensional distances into probabilities and minimizes KL divergence to project points into a low-dimensional space.	- Preserves local structure well - Useful for visualization	- Prone to crowding problem (points collapse) - Difficult to optimize
t-SNE	Improves SNE by using a Student's t-distribution in the low-dimensional space, avoiding crowding and spreading points better.	- Better clustering than SNE - Preserves both local & some global structures - Widely used for data visualization	- Computationally expensive for large datasets - Results vary based on hyperparameters (e.g., perplexity) - Hard to interpret distances globally

- **SOM** is great for clustering but limited in flexibility.
- **SNE** captures local structures but suffers from the crowding problem.
- **t-SNE** improves SNE and is one of the most used techniques for **high-dimensional visualization**.

Dimensionality Reduction II – Learning Outcomes

- **LO1:** understand the basic idea of all the learned dimensionality reduction methods (exam)
 - ❖ E.g., describe the basic idea of the suggested algorithm
 - ❖ E.g., understand the pros and cons of the learned algorithms
 - ❖ E.g., given a dataset, be prepared to follow the selected algorithm to calculate its key steps on the given data
- **LO2:** Implement the learned algorithms for dimensionality reduction (course work)

Assessment hints: Multi-choice Questions (single answer: concepts, calculation etc)

- *Textbook Exercises: textbooks (Programming + Mining)*
- *Other Exercises: <https://www-users.cse.umn.edu/~kumar001/dmbook/sol.pdf>*
- *ChatGPT or other AI-based techs*

Dimensionality Reduction II – Textbook

CHAPTER 7

Dimensionality Reduction

We saw in Chapter 6 that high-dimensional data has some peculiar characteristics, some of which are counterintuitive. For example, in high dimensions the center of the space is devoid of points, with most of the points being scattered along the surface of the space or in the corners. There is also an apparent proliferation of orthogonal axes. As a consequence high-dimensional data can cause problems for data mining and analysis, although in some cases high-dimensionality can help, for example, for nonlinear classification. Nevertheless, it is important to check whether the dimensionality can be reduced while preserving the essential properties of the full data matrix. This can aid data visualization as well as data mining. In this chapter we study methods that allow us to obtain optimal lower-dimensional projections of the data.

- ▶ Data Mining and Machine Learning: Fundamental Concepts and Algorithms M. L. Zaki and W. Meira

Chapter 11

Dimensionality Reduction

There are many sources of data that can be viewed as a large matrix. We saw in Chapter 5 how the Web can be represented as a transition matrix. In Chapter 9, the utility matrix was a point of focus. And in Chapter 10 we

- ▶ Mining of Massive Datasets J. Leskovec *et al*

Appendix

Dimensionality reduction and visualisation is key to understanding the data

Useful for your coursework

There are many ways to do this, with the `sklearn.manifold` library:

<https://scikit-learn.org/stable/modules/manifold.html>

